

**PRAKTIKUM SISTEM OPERASI
MODUL XI
(MEMORI XINU)**



**Universitas
Telkom**

Oleh:

(Tsaqif Hisyam Saputra) (23111042024)

**PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026**

Jurnal Praktikum

Jurnal Praktikum
Sistem Operasi Modul
11: Memori Xinu

Tujuan:

1. Mahasiswa mampu memahami struktur memori pada Xinu.

Catatan:

1. Praktikan wajib untuk screenshot setiap langkah yang dikerjakan hingga tampilan output akhir
2. Untuk soal source code, kumpulkan SS-nya saja
3. Praktikan wajib untuk melakukan screenshot lengkap dengan nama root. Contoh: root@username
4. Berikan identitas nama - nim dalam bentuk comment di Source Code
5. Harap kerjakan secara mandiri, jika tidak paham silahkan bertanya kepada Asisten Praktikum masing-masing. Dilarang mengcopy jawaban dan source code dari teman!

Jurnal

1. **[80 poin]** Buatlah perintah baru bernama **freememory** yang memiliki dua fungsi berikut:
 - a. **[40 poin]** Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. **[40 poin]** Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Output yang diinginkan:

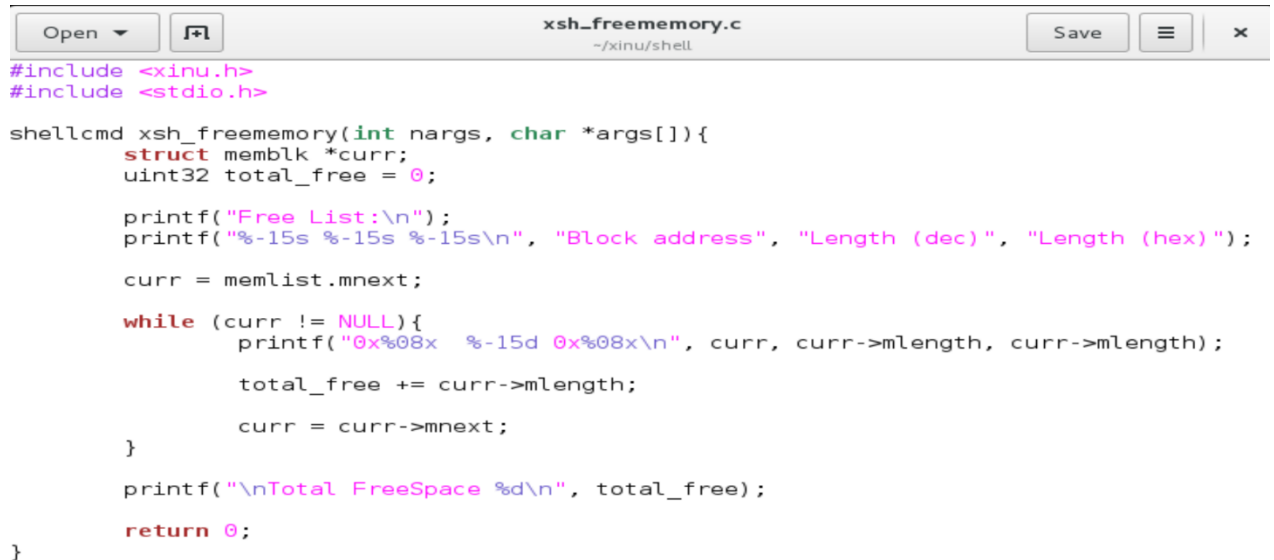
```
Activities [Terminal] Tue 17 May, 18:05
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018f810     -981016      0xffff107e8
0x00100000      5054464      0x004d2000
0x005e4000       57344       0x0000e000
Total FreeSpace 4130792
```

2. [4 poin per subsoal] Jawablah pertanyaan berikut:
- Mengapa Xinu memisahkan data segment dan BSS segment?
 - Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
 - Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
 - Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
 - Apa tantangan utama dalam penggunaan heap di Xinu?

Jawab:

1.



```
Open [icon] xsh_freememory.c ~/xinu/shell Save [icon] [x]
#include <xinu.h>
#include <stdio.h>

shellcmd xsh_freememory(int nargs, char *args[]){
    struct memblk *curr;
    uint32 total_free = 0;

    printf("Free List:\n");
    printf("%-15s %-15s %-15s\n", "Block address", "Length (dec)", "Length (hex)");

    curr = memlist.mnext;

    while (curr != NULL){
        printf("0x%08x  %-15d  0x%08x\n", curr, curr->mlength, curr->mlength);

        total_free += curr->mlength;

        curr = curr->mnext;
    }

    printf("\nTotal FreeSpace %d\n", total_free);

    return 0;
}
```

Logika Pemrograman

Implementasi ini mengandalkan penelusuran struktur data Linked List.

- Akses Titik Awal: Program mengakses variabel global memlist yang bertindak sebagai *head* dari daftar memori bebas.
- Iterasi (Traversing): Menggunakan perulangan while, program berpindah dari satu blok (memblk) ke blok berikutnya menggunakan pointer mnext hingga menemukan nilai NULL.
- Pengolahan Data: Di setiap blok yang dikunjungi, program mengambil data mlength untuk ditampilkan dalam format desimal dan heksadesimal, serta menambahkannya ke variabel akumulator total_free.

```

X I N U
-----2311104024-----

Welcome to Xinu!

xsh $
Demo Memory Xinu
Alokasi 64 bytes di alamat: 18E110
Isi memory ptr1: X I N U
Alokasi 128 bytes di alamat : 18E150
Memory ptr1 telah dibebaskan
Memory ptr2 telah dibebaskan
Done!freememory
Free List:
Block address      Length (dec)      Length (hex)
0x0018e110         -975128          0xffff11ee8
0x00100000         5070848          0x004d6000
0x005e8000         57344            0x0000e000

Total FreeSpace 4153064

```

Analisis Output Terminal

Berdasarkan hasil eksekusi yang dilakukan:

- Struktur Tabel: Output menampilkan alamat memori (seperti 0x0018e110) yang menunjukkan posisi fisik blok tersebut di RAM.
- Variasi Ukuran: Terdapat beberapa blok dengan ukuran berbeda (seperti -975128 dan 5070848), hal ini menunjukkan bahwa memori Xinu telah terbagi-bagi akibat proses alokasi sebelumnya.
- Total Akhir: Hasil Total FreeSpace 4153064 merupakan indikator jumlah memori yang masih tersedia bagi sistem untuk menjalankan proses baru.

```
Activities gedit Mon 11 May, 04:59
Open shell.c ~/xinu/shell

{"memstat", xsh_memstat},
{"ns", xsh_ns},
{"netinfo", xsh_netinfo},
{"ping", xsh_ping},
{"ps", xsh_ps},
{"sleep", xsh_sleep},
{"tee", xsh_tee},
{"udp", xsh_udpdump},
{"udpecho", xsh_udpecho},
{"udpserver", xsh_udpserver},
{"uptime", xsh_uptime},
{"mycmd", xsh_mycmd},
{"namecmd", xsh_namecmd},
{"freememory", xsh_freememory},
{"?", xsh_help}

};
```

```
Activities gedit Mon 11 May, 04:57
Open shprototypes.h ~/xinu/include

/* in file xsh_devdump.c */
extern shellcmd xsh_devdump (int32, char *[]);

/* in file xsh_echo.c */
extern shellcmd xsh_echo (int32, char *[]);

/* in file xsh_ethstat.c */
extern shellcmd xsh_ethstat (int32, char *[]);

/* in file xsh_exit.c */
extern shellcmd xsh_exit (int32, char *[]);

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory (int32, char *[]);

/* in file xsh_help.c */
extern shellcmd xsh_help (int32, char *[]);
```

Tujuan Perintah

Perintah `freememory` dibuat untuk memberikan transparansi terhadap manajemen memori di kernel Xinu. Tujuannya adalah agar praktikan dapat memantau kondisi heap secara real-time, melihat lokasi alamat fisik memori yang sedang bebas, serta memastikan tidak ada kebocoran memori yang signifikan setelah melakukan operasi alokasi.

Langkah Integrasi ke Sistem

Agar perintah ini dapat berjalan, dilakukan tiga tahap integrasi:

1. Pembuatan Source Code: Menulis logika dalam file `xsh_freememory.c`.
2. Registrasi Shell: Mendaftarkan nama perintah di `shell.c` agar shell Xinu mengenali input kata "`freememory`".
3. Kompilasi: Melakukan *rebuild* kernel menggunakan `make` sehingga fungsi baru tersebut dibungkus ke dalam file biner Xinu yang bisa di-boot.

2. Jawaban soal nomor 2:

- **Pemisahan Segment:** Memisahkan variabel yang memiliki nilai awal (Data) dengan yang tidak (BSS) untuk memperkecil ukuran file biner.
- **Alokasi & Dealokasi:** Alokasi memotong isi *free list* (mengurangi memori bebas), sedangkan dealokasi mengembalikan blok ke *free list* (menambah memori bebas).
- **Heap vs Stack:** Heap berisiko mengalami *memory leak* dan fragmentasi jika tidak dikelola secara manual dengan hati-hati.
- **Linked List:** Dipilih karena sangat fleksibel untuk menyambungkan blok-blok memori yang letaknya terpencar di RAM tanpa harus menggeser data lain.
- **Tantangan Xinu:** Keterbatasan sumber daya pada sistem *embedded* dan tidak adanya fitur pembersihan memori otomatis (*garbage collection*).